

Bonnes Pratiques JavaScript — AlerteMarché

Ce document détaille les règles strictes à suivre pour éviter les erreurs JavaScript qui peuvent bloquer le site.

Checklist obligatoire avant chaque commit

- [] Toutes les variables `let / const` sont **déclarées avant utilisation**
- [] Les fonctions `async` gèrent les erreurs avec `try/catch`
- [] Les appels API ont un **fallback** en cas d'échec
- [] Les éléments DOM sont vérifiés avant manipulation (`if (element) { ... }`)
- [] Pas de code bloquant synchrone dans `async function`
- [] Tests manuels effectués sur **tous les pays** (BJ, TG, CI, SN, BF)

Erreurs à éviter absolument

1. Temporal Dead Zone (TDZ) — ReferenceError

 Mauvais :

```
async function initPage() {  
  await loadPremiumAds(); // Utilise premiumAds  
  
  // ... 100 lignes plus bas ...  
  
  let premiumAds = []; // Déclaré APRÈS utilisation ⚠  
}
```

 Correct :

```
async function initPage() {  
  let premiumAds = []; // Déclarer EN PREMIER  
  let premiumIndex = 0;  
  
  await loadPremiumAds(); // Maintenant on peut utiliser  
}
```

Règle : Déclarer toutes les variables **en haut de la fonction** avant tout appel.

2. Promesses non capturées

 Mauvais :

```
async function loadData() {
  const res = await fetch('/api/data'); // Si ça échoue, ça plante TOUT
  const data = await res.json();
}
```

✓ Correct :

```
async function loadData() {
  try {
    const res = await fetch('/api/data');
    if (!res.ok) throw new Error('Fetch failed');
    const data = await res.json();
    return data;
  } catch (error) {
    console.error('Load failed:', error);
    return []; // Fallback vide
  }
}
```

Règle : Toujours `try/catch` les fonctions `async` et prévoir un **fallback**.

3. Manipulation DOM sans vérification

✗ Mauvais :

```
const grid = document.getElementById('tendersGrid');
grid.innerHTML = '...'; // Si grid est null, ça plante
```

✓ Correct :

```
const grid = document.getElementById('tendersGrid');
if (!grid) {
  console.error('Grid element not found');
  return;
}
grid.innerHTML = '...';
```

Règle : Toujours **vérifier** que l'élément existe avant de le manipuler.

4. Dépendances sur variables globales non définies

✗ Mauvais :

```
function myFunction() {
  const country = window.amCode(); // Si amCode() n'existe pas, ça plante
}
```

✓ Correct :

```
function myFunction() {
  const country = (window.amCode && window.amCode()) || 'BJ'; // Fallback
}
```

Règle : Vérifier que les fonctions globales existent avant de les appeler.

Tests obligatoires

Test 1 : Tous les pays fonctionnent

```
# Tester manuellement dans le navigateur
https://www.alertemarche.com/marches-publics?c=BJ
https://www.alertemarche.com/marches-publics?c=TG
https://www.alertemarche.com/marches-publics?c=CI
https://www.alertemarche.com/marches-publics?c=SN
https://www.alertemarche.com/marches-publics?c=BF
```

Vérifier :

- ☒ Le titre affiche le bon pays
- ☒ Le nombre de marchés correspond au pays
- ☒ Aucune erreur dans la console

Test 2 : Console sans erreurs

1. Ouvrir la console (F12)
2. Naviguer sur toutes les pages principales
3. Vérifier : **0 erreur rouge**

Test 3 : Mode déconnecté

Tester que le site fonctionne **sans être connecté** (mode visiteur).

Outils de protection

1. Système de monitoring (error-protection.js)

Ajouter dans **TOUTES** les pages HTML :

```
<script src="js/error-protection.js"></script>
```

Ce script :

- ☒ Capture toutes les erreurs JavaScript
- ☒ Les envoie au backend pour analyse
- ☒ Affiche une modale si trop d'erreurs
- ☒ Évite le blocage silencieux

2. Wrapper pour fonctions critiques

Pour les fonctions `async` critiques (comme `initPage`), utiliser :

```
const safeInitPage = window.AM_ERROR_PROTECTION.wrapAsyncFunction(initPage, 'initPage')
feInitPage();
```



Monitoring backend

Le backend reçoit les erreurs JS via `/api/log-js-error` et les stocke pour analyse.

Tableau de bord (à implémenter) :

- Nombre d'erreurs par jour
- Pages les plus problématiques
- Navigateurs affectés



Workflow de déploiement sécurisé

1. Développement local :

```
bash
npm run dev
# Tester manuellement dans le navigateur
```

2. Tests pré-commit :

```
bash
# Vérifier console : 0 erreur
# Tester tous les pays
```

3. Commit avec message clair :

```
bash
git add .
git commit -m "Fix: [description du problème résolu]"
```

4. Déploiement VPS :

```
bash
scp *.html *.js ubuntu@51.255.166.59:/home/ubuntu/alertemarche/frontend/
```

5. Vérification post-déploiement :

- Ouvrir la production dans le navigateur
- Console : 0 erreur
- Tester un flux complet



En cas de bug en production

Symptôme : “La page ne charge pas”

1. **Console browser** : Noter l'erreur exacte
2. **Rollback immédiat** : Restaurer la dernière version stable
3. **Fix local** : Corriger avec les bonnes pratiques
4. **Tester** : Tous les pays + console

5. Redéployer

Symptôme : “Certains pays ne fonctionnent pas”

Vérifier dans `currentCountry()` que le paramètre URL `?c=XX` est pris en compte AVANT le `localStorage`.

Résumé — Les 5 règles d’or

1. **Déclarer AVANT utiliser** (pas de TDZ)
 2. **try/catch** sur tous les `async`
 3. **Vérifier** les éléments DOM avant manipulation
 4. **Fallback** pour toutes les dépendances externes
 5. **Tester TOUS les pays** avant chaque déploiement
-

Dernière mise à jour : 16 septembre 2026

Auteur : Équipe technique AlerteMarché